

TaxaLikely

Most taxonomic assignments rely on match scores (often percent similarity). Users sometimes confuse a similarity score with the probability that a match is correct, but a 99% similarity between an observation and a reference is not the same as a 99% chance that the observation is that species – several species may be 99% similar. To link scores to probabilities, match scores must first be converted to likelihoods. TaxaLikely does this by fitting statistical models that compare within-species scores to between-species scores in a reference library. This process can also flag mislabeled references (e.g., a sequence that matches other species better than its own). Removing such errors improves assignment accuracy. TaxaLikely also models the expected score profile of species absent from the reference library, reducing false positives caused by missing references. By replacing arbitrary score thresholds with continuous likelihoods, TaxaLikely avoids both overconfident assignments and unnecessary loss of taxonomic resolution. Calibrated likelihoods can then be multiplied by priors (see TaxaExpect) to generate Bayesian posterior probabilities for each hypothesized assignment. Part of the TaxaID ecosystem.

Overview

TaxaLikely trains a hierarchical model on reference-vs-reference match scores (DNA percent identity, image similarity, acoustic scores) and applies it to convert per-observation scores into likelihoods. Supports three hypothesis types:

- **H1 (Known species)** – target taxon is in the reference database
- **H2 (Unreferenced species)** – a congener absent from the reference
- **H3 (Unreferenced genus)** – a taxon from a different genus entirely

Installation

```
# Requires TaxaTools (foundation package)
devtools::install("path/to/TaxaTools")
devtools::install("path/to/TaxaLikely")

# Bioconductor dependency for reference matrix building
BiocManager::install("DECIPHER")
```

Quick Start

```
library(TaxaLikely)

# 1. Fetch reference sequences from NCBI
reference_df <- fetch_reference_sequences(
  taxa = c("Fundulidae", "Gobiidae"),
  barcode_term = "12S",
  rank = "family"
)

# 2. Build pairwise distance matrix
ref_matrix <- build_sequence_matrix(reference_df)
```

```

# 3. Flag mislabeled references
errors <- flag_reference_errors(ref_matrix)

# 4. Train the likelihood model
model <- train_likelihood_model(ref_matrix)

# 5. Apply model to match data (from TaxaMatch)
result <- evaluate_likelihoods(match_df, model)
likelihoods <- result$likelihoods
# Columns: observation_id, taxon_name, hypothesis_type,
#           score_likelihood, score_likelihood_mean, score_likelihood_sd,
#           score_likelihood_cov

```

Loading Pre-built Reference Databases

TaxaLikely supports several common reference database formats directly.

CRABS (recommended for eDNA barcode databases)

CRABS (Creating Reference databases for Amplicon-Based Sequencing) is a widely-used eDNA reference database builder. Its internal format is a headerless tab-delimited file with 11 fixed columns (accession through sequence). Load it with:

```

ref <- read_crabs_output(
  file           = "mifish_12S_crabs.tsv",
  rank_system    = c("family", "genus", "species"), # or NULL to auto-detect
  max_n_bases    = 250,          # optional: drop unusually long sequences
  dereplicate    = TRUE          # optional: collapse identical seqs within species
)

```

CRABS + TaxaLikely are complementary. CRABS excels at bulk retrieval, length filtering, primer trimming, and exact-sequence dereplication at database-build time. TaxaLikely catches what CRABS cannot: mislabeled sequences where the species annotation is wrong but the sequence itself is valid. These produce within-species distances that look like between-species distances and inflate false-positive rates. Always follow `read_crabs_output()` with `flag_reference_errors()` on the alignment matrix to catch these:

```

ref_matrix <- build_sequence_matrix(ref)
errors     <- flag_reference_errors(ref_matrix)
clean_mat  <- remove_flagged_references(ref_matrix, errors)
model      <- train_likelihood_model(clean_mat)

```

FASTA + separate taxonomy table

`read_reference_fasta()` accepts a FASTA file and a separate taxonomy source. Two input formats are supported:

Data frame (custom databases, CRUX, GenBank dumps):

```
tax <- data.frame(
  composite_id = c("ACC001", "ACC002"),
  family = c("Fundulidae", "Atherinopsidae"),
  genus = c("Fundulus", "Atherinops"),
  species = c("Fundulus parvipinnis", "Atherinops affinis")
)
ref <- read_reference_fasta("sequences.fasta", taxonomy = tax,
  rank_system = c("family", "genus", "species"))
```

Taxonomy TSV file (QIIME2 / RESCRIPT / SILVA / MIDORI2): supply `taxonomy_file` instead of `taxonomy`. Both prefix-style (`k__Kingdom;p__Phylum;...`) and positional (`Kingdom;Phylum;...`) formats are auto-detected. Header rows are skipped automatically.

```
# Works with QIIME2 taxonomy artifacts, RESCRIPT output, or MIDORI2 files
ref <- read_reference_fasta(
  "sequences.fasta",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "taxonomy.tsv"
)
```

Reference Databases at a Glance

The table below summarises the databases commonly used with TaxaLikely, the data types they cover, and the recommended loading path.

Database	Focus	Typical size	Loading path	Notes
NCBI GenBank	Universal	API (no local file)	<code>fetch_reference_sequences()</code>	Primary API query; best for targeted eDNA marker retrieval
CRABS output	eDNA amplicons	Varies	<code>read_crabs_output()</code>	CRABS handles bulk QC; TaxaLikely adds mislabel detection
SILVA SSU	16S / 18S / 23S rRNA	~1.3 GB	<code>subset_local_database()</code>	Primary database for microbial amplicon eDNA; ~510 k sequences
MIDORI2	COI + nuclear markers	~4.4 GB (COI)	<code>subset_local_database()</code>	Best for metazoan COI and nuclear eDNA markers
GTDB	Bacterial / archaeal 16S	~500 MB (16S subset)	<code>subset_local_database()</code>	Prokaryotic genomic taxonomy; differs from NCBI; export via QIIME 2
Greengenes2	16S rRNA	~1.2 GB	<code>subset_local_database()</code>	2012 release; GTDB-derived taxonomy; export from QIIME 2

Database	Focus	Typical size	Loading path	Notes
RDP	16S / 28S rRNA	~200 MB	<code>read_reference_fasta()</code>	Smaller than SILVA; reformat lineage file to 2-col TSV first

SILVA, MIDORI2, GTDB, and Greengenes2 are distributed as bulk downloads (multi-gigabyte FASTA + taxonomy files). `subset_local_database()` streams these files rather than loading them into memory, so filtering to the genera or families relevant to your site is fast regardless of total database size. CRABS can also download from SILVA and BOLD directly and produce its own internal format, which `read_crabs_output()` handles. For NCBI, use `fetch_reference_sequences()` or `build_site_reference()` to fetch only the taxa you need.

Subsetting a Large Local Database

`subset_local_database()` filters any large FASTA + taxonomy file to a user-supplied taxon list. The taxonomy file is parsed first to identify matching sequence IDs; the FASTA is then streamed, so peak memory scales with the number of matching sequences rather than the total database size.

```
library(TaxaLikely)

# --- SILVA SSU example: filter to two fish families -----
ref <- subset_local_database(
  fasta_path = "SILVA_138.1_SSURef_NR99.fasta.gz",
  taxa       = c("Fundulidae", "Gobiidae"),
  rank       = "family",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "silva_taxonomy.tsv"
)

# --- MIDORI2 COI example: filter to a genus; discard sequences > 700 bp -----
ref <- subset_local_database(
  fasta_path = "MIDORI2_UNIQ_NUC_GB260_COI_QIIME.fasta",
  taxa       = "Thunnus",
  rank       = "genus",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "MIDORI2_UNIQ_NUC_GB260_COI_QIIME_taxon.tsv",
  max_n_bases = 700L,
  require_species = TRUE
)

# Both return a reference_df - continue with the standard workflow:
ref_matrix <- build_sequence_matrix(ref)
errors     <- flag_reference_errors(ref_matrix)
model      <- train_likelihood_model(ref_matrix)
```

`subset_local_database()` accepts plain-text and .gz-compressed FASTA files. Taxonomy strings in SILVA, MIDORI2, GTDB, and Greengenes2 prefix (d__, p__, ...) or positional (no-prefix semicolon) formats are both auto-detected. For GTDB or Greengenes2 sources, export the FASTA + taxonomy TSV from QIIME 2 with `qiime tools export` before calling the function.

Building a Site-Specific Reference Library

`build_site_reference()` is a one-call wrapper for building a curated local reference matched to your site's expected taxa. Starting from a list of genera or families (e.g., from `TaxaExpect::build_priors()`), it fetches sequences from NCBI, audits which described species have no barcode sequences at all (the unreferenced gap list), and exports a FASTA + taxonomy TSV ready for BLAST or QIIME 2.

```
library(TaxaLikely)

# Genera expected at your site (from TaxaExpect or user-supplied)
site_genera <- c("Fundulus", "Gambusia", "Lepomis", "Micropterus")

lib <- build_site_reference(
  taxa          = site_genera,
  barcode_term  = "MiFishU",
  output_dir    = "site_reference/",      # writes reference.fasta + taxonomy TSV
  max_date      = "2024/12/31"           # reproducible: fix GenBank state
)

lib$unreferenced # species with NO barcode in NCBI -- cannot be detected
lib$census       # per-genus completeness summary

# Pass gap list to TaxaAssign for ghost hypothesis handling
unreferenced_result <- TaxaAssign::suggest_unreferenced_species(
  match_df      = match_data,
  unreferenced_taxa = lib$unreferenced
)

# Train model on the curated reference
model <- train_likelihood_model(
  build_sequence_matrix(lib$reference_df)
)
```

To export any `reference_df` to FASTA without the full download pipeline:

```
write_reference_fasta(
  reference_df,
  file          = "my_reference.fasta",
  taxonomy_file = "my_taxonomy.tsv"    # QIIME2-compatible; reload with read_reference_fasta()
)
```

Key Functions

Reference acquisition and export (DNA): - `build_site_reference()` – one-call site-specific reference builder: taxa list → NCBI fetch → coverage audit → FASTA export - `subset_local_database()` – stream a large local FASTA + taxonomy file (SILVA, MIDORI2, GTDB, Greengenes2) and extract sequences for a user-supplied taxon list; memory scales with matches, not database size - `write_reference_fasta()` – export any `reference_df` to FASTA + optional taxonomy TSV (round-trippable with `read_reference_fasta()`) - `read_crabs_output()` – load a CRABS internal-format database (taxonomy embedded; no separate file needed) - `fetch_reference_sequences()` – download from NCBI by taxon + barcode marker - `read_reference_fasta()` – load local FASTA + data-frame taxonomy (or `taxonomy_file` TSV for QIIME2/RESRIPT/MIDORI2)

Model training (DNA): - `build_sequence_matrix()` - pairwise distance matrix via DECIPHER; required for `flag_reference_errors()` and `train_likelihood_model()` - `flag_reference_errors()` - detect mislabeled references - `train_likelihood_model()` - fit hierarchical Bayesian model

Inference: - `evaluate_likelihoods()` - convert match scores to likelihoods using a trained model - `filter_top_hypotheses()` - keep finest-rank candidates per query - `unreferenced_candidates()` - expand a consensus assignment with H2/H3 placeholder rows (no model required; used in no-score and acoustic/image pathways) - `assign_scores()` - set `score_likelihood` values: uniform ("none"), ratio-normalised ("probability"), softmax ("similarity_softmax"), or prepare for the bivariate-normal model ("similarity") - `compute_likelihoods()` - high-level wrapper: `unreferenced_candidates()` → `assign_scores()` → `model_likelihoods()` (DNA scored pathway)

Reference QC: - `audit_barcode_coverage()` - find unreferenced species (no barcode sequence; eDNA/DNA only) - `audit_acoustic_coverage()` - find plausible species absent from classifier's known list (acoustic/image) - `audit_reference_coverage()` - taxonomic completeness check - `apply_coverage_constraints()` - suppress H2 for fully-sampled genera - `remove_flagged_references()` - clean match data using error flags (DNA only; see @section Scope: in docs for acoustic/image guidance)

Coverage quality filtering: - `calibrate_coverage_filter()` - sweep a grid of coverage thresholds and return breadth vs H1/H2 discrimination metrics (including Youden's J) to identify the Pareto-optimal threshold - `coverage_threshold()` - quick quantile-based shortcut: set the threshold so that a target fraction (default 95%) of pairs are retained, with automatic snapping to the nearest grade boundary for categorical (acoustic) coverage

Diagnostics and reporting: - `interpret_model()` - summarize trained model parameters - `report_likelihood()` - generate report section for `assemble_report()`

When to use `flag_reference_errors()`

`flag_reference_errors()` detects mislabeled sequences in a DNA reference matrix. The function's applicability depends on the data source:

Reference source	Use <code>flag_reference_errors()</code> ?	Notes
NCBI nucleotide (via <code>fetch_reference_sequences()</code>)	Yes — recommended	NCBI has well-known curation issues: automated submissions, misidentified vouchers, contamination. Use routinely.
Curated libraries (CRUX, custom expert-built FASTA)	Optional	Lower mislabeling rate than NCBI, but flagging is still worth running. If your library has a quality column, use that filter instead.
Xeno-canto bird sounds (acoustic)	No	Xeno-canto is expert-curated; species identity mislabeling is rare. The dominant noise source is recording conditions (distance, background), not wrong species. Use <code>quality = c("A", "B")</code> in <code>fetch_reference_recordings()</code> instead. Hard-case recordings flagged by the mislabel detector may be legitimate.

Reference source	Use <code>flag_reference_errors()</code> ?	Notes
Camera trap images (Animl/SpeciesNet)	Untested — potentially useful	Camera trap ground-truth labeling has different error modes from DNA (occlusion, blur, multiple animals, handler setup). The score-distribution mislabel signal should still be informative in principle — a consistently low-scoring “within-species” pair is suspect regardless of data type — but systematic evaluation has not been done. See the Image Workflow section below.

Reference Coverage Quality Filtering

Match scores (`p_match`) measure how similar two sequences or audio clips are, but they do not capture *how much* of each observation contributed to that score. A 99% DNA identity computed from a 50 bp fragment of a 600 bp barcode is far less reliable than the same identity computed from a 580 bp overlap — yet both produce the same score. Similarly, a high BirdNET confidence from a poor-quality recording (heavy rain, distant call) carries less information than the same score from a quiet, close-range recording.

`build_sequence_matrix()` attaches a **coverage** column to its output: *alignment coverage* is the number of positions where both sequences contribute a non-gap character, divided by the shorter unaligned sequence length. Values near 1.0 indicate nearly complete overlap; values near 0.0 indicate highly gappy or partial alignments.

Choosing a threshold: `calibrate_coverage_filter()`

`calibrate_coverage_filter()` sweeps a grid of thresholds and returns a data frame of per-threshold metrics:

Metric	Description
<code>breadth</code>	Fraction of unique queries (<code>id_x</code>) surviving the filter
<code>h1_retention</code>	Fraction of within-species (H1) pairs retained
<code>h2_retention</code>	Fraction of cross-species (H2/H3) pairs retained
<code>youden_j</code>	<code>h1_retention - h2_retention</code> — Youden’s J; maximised at the Pareto-optimal threshold
<code>discrimination</code>	<code>h1_retention / h2_retention</code> — ratio form; useful for log-scale plots
<code>mean_h1_score</code>	Mean <code>p_match</code> of retained H1 pairs

Youden’s J (sensitivity + specificity − 1 in ROC terminology) is the primary selection criterion. It is bounded [−1, 1], equals 0 at the no-filter baseline, treats H1 retention and H2/H3 exclusion symmetrically, and has no divide-by-zero edge case. The threshold that maximises J retains the most within-species signal while removing the most cross-species noise.

The **discrimination** column (ratio form) conveys the same information on a multiplicative scale and is useful when plotting on a log axis.

Note for acoustic data: Xeno-canto quality grade is a property of the recording, not the detection pair. Every pair produced from the same recording — both H1 and H2/H3 — carries the identical **coverage**

value. Filtering therefore removes entire recordings, excluding H1 and H2/H3 pairs in equal proportion. For acoustic data, `youden_j` and `discrimination` will be near-flat; use `breadth` and `mean_h1_score` as the primary guides. The function detects categorical coverage and emits a message in this case.

```
ref_matrix <- build_sequence_matrix(reference_df)

cal <- calibrate_coverage_filter(ref_matrix)

# Pareto-optimal threshold: maximises Youden's J
best <- cal[which.max(cal$youden_j), ]
cat("Threshold:", best$threshold,
    "| J =", round(best$youden_j, 3),
    "| breadth =", round(best$breadth, 3), "\n")

# Visualise the trade-off
plot(cal$breadth, cal$youden_j, type = "b",
     xlab = "Breadth (fraction of queries retained)",
     ylab = "Youden's J",
     main = "Coverage filter calibration")
abline(v = best$breadth, lty = 2, col = "red")

# Apply before training
ref_filtered <- ref_matrix[ref_matrix$coverage >= best$threshold, ]
model <- train_likelihood_model(ref_filtered)
```

Quick alternative: `coverage_threshold()`

`coverage_threshold()` selects the threshold that retains a target fraction of pairs by quantile — no H1/H2 structure required:

```
# Retain the best 90% of pairs by alignment coverage (discard bottom 10%)
thresh <- coverage_threshold(ref_matrix, keep_frac = 0.90)
ref_filtered <- ref_matrix[ref_matrix$coverage >= thresh, ]
model <- train_likelihood_model(ref_filtered)

# Acoustic: snaps to nearest Xeno-canto quality grade boundary
thresh <- coverage_threshold(ref_acoustic, keep_frac = 0.80)
# Message: "Snapping threshold 0.72 -> 0.80 (retains 83.1% of pairs, requested 80.0%)"
```

Statistical Design

TaxaLikely models the joint distribution of two features – the logit-transformed match score (absolute fit) and the gap to the best alternative (relative uniqueness) – as a bivariate normal for each hypothesis type:

- **Score + gap features:** Raw scores are logit-transformed to an unbounded domain; the gap is computed in logit space so that differences near 100% are amplified appropriately
- **Bivariate normal likelihood:** The joint (score, gap) density captures interactions – a small gap is more tolerable when the score is very high
- **Empirical Bayes shrinkage:** Per-species parameters are shrunk toward the global mean (Efron and Morris 1973), with weight inversely proportional to sample size, preventing poorly sampled species from having unreliable estimates

- **H2/H3 offset distributions:** Unreferenced species and genus hypotheses use the H1 distribution shifted left by learned delta offsets, estimated from cross-species match scores in training data
- **Perfect-match anchoring:** Synthetic 100% match pseudo-data prevent the “perfection penalty” where the Gaussian density peaks below 100%
- **Monte Carlo uncertainty:** Score perturbation across simulations yields `score_likelihood_mean` and `score_likelihood_sd`, measuring sensitivity to measurement noise
- **Alignment coverage filter (optional):** A `min_coverage` threshold can be passed to `evaluate_likelihoods()` to drop low-coverage candidates before scoring. Use `calibrate_coverage_filter()` on the training matrix to find the Pareto-optimal threshold (maximises Youden’s J: H1 retention minus H2 retention). Acoustic and image data can supply a categorical quality column (e.g. Xeno-canto grade) encoded as an integer for the same pre-filter.
- **Coverage-adjusted likelihood (`score_likelihood_cov`):** When the match object contains a `coverage` column, `evaluate_likelihoods()` produces a second point-estimate column `score_likelihood_cov` alongside the standard `score_likelihood`. For each H1 candidate the model sigma is widened by $1 / \sqrt{\text{coverage}}$ before the likelihood is evaluated — grounded in binomial sampling theory: the standard error of a proportion estimated from N aligned positions scales as $1 / \sqrt{N}$, and `N_aligned = coverage × N_total`. H2 and H3 sigmas are global fixed parameters and are not inflated.

Widening sigma changes the **point estimate** of the likelihood (not its SD), because it changes the density value returned by the bivariate normal at the observed score. The direction of the change depends on where the observed score falls relative to the H1 mean:

- **Score near the H1 mean (good match):** the density at the peak decreases as the distribution flattens → `score_likelihood_cov < score_likelihood`. This is the primary intended effect: a 98% match at 60% coverage is penalised relative to the same score at full coverage, reducing overconfidence in partial alignments.
- **Score far below the H1 mean (poor cross-species match):** the distribution flattens into a wider tail, increasing the density at that low score → `score_likelihood_cov > score_likelihood`. This is a secondary effect: the model is saying “with low coverage I cannot rule out H1 as strongly.” These candidates are already ranked low and the effect rarely changes downstream assignments.

The crossover is at exactly ± 1 sigma from the mean — a useful check is whether the best H1 candidate is a good match (expect negative delta) or a poor match (expect positive delta).

Why this is not a model parameter: Reference-vs-reference training pairs (from `build_sequence_matrix()`) are nearly always full-length alignments — same-species sequences share the same amplicon and align completely. There is therefore no within-H1 coverage variation in the training data from which to estimate a coverage-score relationship. The $1 / \sqrt{\text{coverage}}$ inflation is applied as a principled prior at inference time rather than a fitted parameter.

How to use it: Pass `score_likelihood_cov` to `TaxaAssign::compute_posterior()` instead of `score_likelihood` to apply the adjustment. Compare the two columns to find queries where coverage meaningfully shifts the likelihood ratios:

```
likelihoods |>
  dplyr::filter(
    hypothesis_type == "specific_candidate",
    abs(score_likelihood_cov - score_likelihood) > 0.05
  ) |>
  dplyr::arrange(dplyr::desc(abs(score_likelihood_cov - score_likelihood)))
```

When coverage is absent from the match object or equals 1 for all candidates, `score_likelihood_cov` is identical to `score_likelihood`.

For a detailed treatment of the statistical framework, feature engineering, hypothesis definitions, parameter estimation, and reference quality control, see `inst/TaxaLikely_supplemental_methods.md`.

Acoustic and Image Workflows

For acoustic (BirdNET) and image (SpeciesNet, Animl, iNaturalist CV) data, TaxaLikely works as a **post-classifier** layer. Users bring classifier output files that already contain candidates and confidence scores — TaxaLikely converts those scores to likelihoods and expands the candidate set using TaxaExpect priors.

There are two entry points depending on how many candidates the classifier provides per observation:

Multiple candidates per observation (e.g., SpeciesNet top-k)

When the classifier returns ranked candidate lists (top-k species with scores), the match data maps directly to the scored pathway. Use `evaluate_likelihoods()` exactly as in the DNA workflow:

```
library(TaxaLikely)
library(TaxaMatch)

# Read SpeciesNet / Animl top-k output → canonical match object
# (scientific names required; use TaxaTools::common_to_scientific()
# if your classifier outputs common names)
match_df <- read_animl_output("animl_results/", min_confidence = 0.3) |>
  subset(!species %in% c("empty", "human", "vehicle"))

# Check coverage: which plausible species are absent from classifier's
# known species list?
census <- audit_acoustic_coverage(
  plausible_species = c("Ursus americanus", "Puma concolor", "Cervus canadensis"),
  reference_species = speciesnet_species_list
)
census$unreferenced # species the classifier has never seen

# Evaluate likelihoods (requires a trained model or use no-score pathway)
result <- evaluate_likelihoods(match_df, image_model)
likelihoods <- result$likelihoods
```

Single best candidate with a score (e.g., BirdNET top-1)

`assign_scores()` anchors H2/H3 likelihoods at the **median H1 likelihood across all candidates for that observation**. With only one H1 row per observation (top-1 output), that median is always 1.0 — so the confidence score has no effect and all three rows receive `score_likelihood = 1.0`.

Recommendation: keep BirdNET's full ranked output (multiple candidates per segment) and use `score_type = "probability"`. BirdNET's default output already includes ranked species with confidence scores:

```
library(TaxaLikely)

# BirdNET full output - multiple candidates per clip, each with confidence
# (use TaxaTools::common_to_scientific() first if output has common names)
birdnet_df <- data.frame(
```

```

observation_id = c("clip_001", "clip_001", "clip_001"),
taxon_name     = c("Melospiza melodia", "Passerella iliaca", "Junco hyemalis"),
taxon_name_rank = c("species", "species", "species"),
family         = c("Passerellidae", "Passerellidae", "Passerellidae"),
genus          = c("Melospiza", "Passerella", "Junco"),
species        = c("Melospiza melodia", "Passerella iliaca", "Junco hyemalis"),
score_original = c(0.87, 0.09, 0.04) # BirdNET confidence scores
)

# Step 1: add H2/H3 placeholder rows
hyp_df <- unreferenced_candidates(birdnet_df,
                                rank_system = c("family", "genus", "species"))

# Step 2: ratio-normalise; top candidate = 1.0, lower ones proportionally less
likelihoods <- assign_scores(hyp_df, score_type = "probability")

# Feed directly to TaxaAssign:
posteriors <- TaxaAssign::compute_posterior(likelihoods, priors_df = my_priors)

```

If you only have top-1 BirdNET output and cannot recover the full ranked list, use `score_type = "none"` — posteriors will be proportional to TaxaExpect priors alone (the confidence score is ignored).

Common names in classifier output

If your classifier outputs common names rather than scientific names, convert them first with `TaxaTools::common_to_scientific()`:

```

library(TaxaTools)

sci <- common_to_scientific(
  common_names = c("White-tailed Deer", "Raccoon", "Wild Turkey"),
  taxon_group  = "mammals and birds",
  location     = "Eastern USA"
)
sci[, c("common_name", "scientific_name_verified", "verified")]

```

No-Score Pathway

When match scores are unavailable — morphology-based identifications, expert IDs, upranked consensus outputs from a previous run, or any source that yields a taxon name but no similarity score — `unreferenced_candidates()` + `assign_scores(score_type = "none")` builds a degenerate likelihood object that bypasses `TaxaMatch` and `evaluate_likelihoods()` entirely.

Two placeholder rows are added per observation: - **H2 (unreferenced_species)** — a placeholder for any species in the same genus not in the reference database - **H3 (unreferenced_genus)** — a placeholder for any genus in the same family not in the reference database

All `score_likelihood` values are set to 1.0 (uniform), so posteriors are proportional to TaxaExpect priors. No `priors_df` is needed at this stage — priors are joined later by `TaxaAssign::join_priors()`.

```

library(TaxaLikely)

```

```

# Consensus taxon assignments with no match scores
consensus_df <- data.frame(
  observation_id = c("obs1", "obs2", "obs3"),
  taxon_name     = c("Salmo salar", "Salvelinus", "Cyprinus carpio"),
  taxon_name_rank = c("species", "genus", "species"),
  family         = c("Salmonidae", "Salmonidae", "Cyprinidae"),
  genus          = c("Salmo", "Salvelinus", "Cyprinus"),
  species        = c("Salmo salar", NA_character_, "Cyprinus carpio")
)

# Step 1: add H2/H3 placeholder rows
hyp_df <- unreferenced_candidates(
  match_df = consensus_df,
  rank_system = c("family", "genus", "species")
)

# Step 2: set all score_likelihood = 1.0
likelihoods <- assign_scores(hyp_df, score_type = "none")

# Three rows per observation (H1 + H2 + H3); all score_likelihood = 1.0.
# Feed directly to TaxaAssign; posteriors will be proportional to priors:
posteriors <- TaxaAssign::compute_posterior(likelihoods, priors_df = my_priors)

```

When to use this pathway

- Morphology or expert IDs that have no match scores
- Consensus outputs upranked to genus or family (score discrimination failed; use priors to probe which species is most likely)
- Stability checks: does an unreferenced congener have a higher prior than the consensus species? If so, flag for review.
- Mixed datasets: run `evaluate_likelihoods()` on scored observations and `unreferenced_candidates()` + `assign_scores()` on unscored ones, then `dplyr::bind_rows()` the outputs before calling `compute_posterior()`.

See `inst/workflows/6_no_score_pathway_workflow.R` for a full example.

Vignettes

- Score to Likelihood – full workflow

Part of TaxaID

TaxaLikely receives match data from TaxaMatch and produces calibrated likelihoods for TaxaAssign (posterior computation).

Ecosystem: TaxaMatch -> **TaxaLikely** -> TaxaAssign

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$)
- TaxaTools (foundation package, installed first)
- DECIPHER and Biostrings (Bioconductor; required for `build_sequence_matrix()` only)
- rentrez and xml2 (for NCBI reference fetching and coverage auditing)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic).